

1. Research question and complete workflow

The project asks whether deep learning can estimate an economically meaningful stochastic discount factor (SDF), and whether replacing macroeconomic LSTM encoders with causal Transformers changes portfolio performance. The original framework connects neural networks to conditional asset-pricing restrictions. Our extensions examine performance across time and economic conditions, combinations of architectures, and sampling uncertainty.

The workflow is: prepare firm and macro data; define chronological train, validation and test samples; learn economic states from macro history; produce stock weights; form a return factor and SDF; calculate pricing moments; update trainable parameters; choose checkpoints using validation; combine trained members; evaluate fixed portfolios on test data; analyze rolling windows, regimes and uncertainty.

There are two meanings of end-to-end. The research pipeline connects data to reported results. End-to-end learning means that the economic loss backpropagates through the portfolio-weight network and its macro encoder. The state representation is learned to serve the pricing objective, rather than estimated independently and permanently fixed.

Main distinction: subsequent realized returns enter the training loss, while portfolio weights at time t must depend only on information available at time t . Test data evaluate the learned model and must not enter gradient updates or checkpoint selection.

2. Data preparation and chronological samples

The stock panel contains 46 firm characteristics and subsequent excess returns. Macroeconomic history contains 178 variables. The pipeline aligns dates and asset observations, handles missing observations using the project's masks and preprocessing, and preserves asset identity when calculating time-average pricing moments. Characteristic inputs in the extension panel are bounded around minus 0.5 to plus 0.5.

The chronological split is training 1967-1986, validation 1987-1991, and test 1992-2016. These spans contain 240, 60 and 300 months respectively. A large cross-section of stocks provides many firm observations, but the macro encoder still has only one common economic observation per month. This limits the number of distinct historical macro sequences available for learning.

Training data drive parameter updates. Validation data select checkpoints or tuning choices. Test data assess the chosen strategy. Preprocessing and information timing must prevent future observations from entering earlier decisions. In the completed matched extension, validation and test macro states use available historical prefixes, with causal processing.

Our historical replication reconstructs the pricing framework and benchmark portfolios. It provides an economic reference, but it differs from the later matched implementation in training and ensemble conventions. For that reason, replication versus extension LSTM is not a clean one-variable experiment.

3. Macro encoders and portfolio weights

For the SDF network, macro history is compressed into four learned states. For the adversarial conditional network, a separate encoder produces 32 states. Separate encoders permit different representations for producing pricing weights and identifying difficult conditional moments.

The LSTM updates its hidden state sequentially month by month. Its recurrence summarizes prior information in a compact memory. The Transformer projects macro observations into representations, adds positional information, and uses causal attention to combine relevant available months. In the completed extension, each Transformer has two layers and four attention heads, with internal width 32 and feedforward width 64. A causal mask prevents future-month access.

The pricing weight network combines a stock's 46 characteristics with the four pricing macro states. Two hidden layers of 64 units produce a scalar stock score or raw portfolio weight. These weights can represent long and short exposures. The conditional network combines firm characteristics with the 32 adversary states and produces eight bounded instruments through a tanh output in the matched implementation.

Code 08 replaces both macro LSTMs with Transformers while retaining economic output dimensions and the pricing framework. This tests a change in the temporal representation, rather than substituting a language model or generating text.

4. Economic objective and backpropagation

Let $x(i,t)$ denote stock characteristics and $h(t)$ the learned macro state. The pricing network produces a weight $w(i,t) = f(x(i,t), h(t))$. The raw factor is $F(t+1) = \sum_i w(i,t) \times R \text{ excess}(i,t+1)$. The model defines $M(t+1) = 1 - F(t+1)$.

The economic condition for excess returns is $E[M(t+1) \times R \text{ excess}(i,t+1) \mid \text{information at } t] = 0$. Conditional instruments $g(i,t)$ convert this into unconditional empirical moments: average M times excess return times g for each asset over its observed months. Squared moments form the pricing loss, with observation-count weighting. The matched code uses count ratios based on the largest available asset count and averages across instruments and assets.

A small loss means the selected sample moments are close to zero. It does not automatically prove every conditional pricing restriction holds, nor guarantee a high future Sharpe. Raw weights used for economic training also differ from the gross-normalized portfolio return convention used in reported performance.

In a pricing-model update, backpropagation differentiates this loss through factor returns, stock weights, the feedforward network and macro encoder. Adam changes their parameters. The weights affect the factor, the factor affects the SDF, and the SDF affects pricing errors. This continuous chain is the central end-to-end learning mechanism.

Sharpe is primarily a performance statistic here. Stage 3 also uses validation raw-factor Sharpe to select a checkpoint, but the gradient update minimizes economic pricing loss. Distinguish selection criterion from training objective.

5. Three-stage adversarial learning and checkpoints

Stage 1 fits an initial pricing model using unconditional moments, equivalent to using an instrument of one. Update the SDF encoder and weight network. In the matched implementation, select a checkpoint with low validation unconditional pricing loss after the warm-up.

Stage 2 freezes the pricing side and trains the conditional encoder and instrument network to maximize pricing errors. The adversary searches for economic conditions or test assets where the current SDF has difficulty. Maximizing a loss is implemented by minimizing its negative. Select the adversary using the training score under the audited convention.

Stage 3 freezes the selected adversary and updates the SDF encoder and pricing network to reduce the resulting conditional moments. The matched implementation selects a checkpoint using the highest validation raw-factor Sharpe after warm-up. The test sample does not select the checkpoint.

The alternation prevents both sides from optimizing in the same direction. A strong adversary creates a demanding pricing problem; the SDF model improves against that fixed demand. It is an adversarial moment-learning framework, rather than a generator producing synthetic return observations.

The historical replication restores a selected Stage 2 adversary but uses final Stage 1 and Stage 3 pricing weights. Its checkpoint rules therefore differ from the matched extension. Matching nominal epoch counts alone does not reproduce every training decision.

6. Seeds, schedules and ensemble construction

A seed controls initialization and other random draws. Multiple seeds produce multiple independently initialized members of the same architecture. The original audited setup uses nine members. Our historical replication and matched extensions use seeds 42 through 50. The exact author seed values should not be inferred from our seed list.

The audited author schedule has 256 Stage 1 epochs, 64 Stage 2 epochs and 1024 Stage 3 epochs, with four optimizer passes and a warm-up setting of 64. The historical replication uses those nominal stage counts but one optimizer step per epoch in its reusable training loop. The completed matched extension uses 16, 4 and 48 epochs with four optimizer passes and a warm-up setting of four.

Both new architectures receive the same reduced schedule. Eighteen members complete: nine LSTM and nine Transformer. The matched implementation uses Adam with learning rate 0.001. It is a PyTorch port, whereas the original author implementation uses TensorFlow. Same seeds across frameworks do not guarantee identical random draws.

In the matched extension, average raw stock weights across members and then normalize each month's ensemble weights by gross exposure, the sum of absolute weights. Apply these normalized weights to subsequent excess returns to obtain the reported portfolio series. This is different from averaging individual Sharpe ratios. Differences from the historical ensemble construction further limit direct causal interpretation of replication-versus-extension gaps.

Shortened training reduces computation but may affect convergence. Equal budgets make the architecture experiment controlled in effort, while not ensuring each architecture is optimally tuned. A full-schedule experiment was interrupted and is not a completed comparison.

7. Test evaluation and extension pipeline

Monthly Sharpe is average monthly portfolio excess return divided by its monthly standard deviation, using population standard deviation in the current comparison. Annualized Sharpe multiplies the monthly ratio by square root of 12 under the project's reporting convention. Mean return, volatility and maximum drawdown provide complementary evidence.

Completed full-test monthly Sharpe is approximately 0.750 for the author factor, 0.612 for our replicated GAN, 0.459 for the matched extension LSTM, 0.304 for Transformer and 0.463 for the equal-weight return blend. The original paper values are published benchmarks, while additional comparisons use saved return series.

Code 09 evaluates saved strategies within complete 36- and 60-month windows. Three hundred test months yield 265 and 241 overlapping windows. This is rolling evaluation, not retraining. Mean rolling Sharpe differs from full-test Sharpe because it averages local ratios rather than constructing one ratio across all months.

Code 10 labels economic conditions. There are 274 expansion and 26 recession months. Lagged market volatility with a training-set threshold gives 123 high-volatility and 177 low-volatility months. These classifications evaluate fixed strategies; they do not fit separate regime-specific models. Code 11 combines these analyses and examines the equal-weight architecture blend.

Code 012 applies paired circular block bootstraps to fixed saved returns. Full-test analysis uses 10,000 replicates and primary twelve-month blocks, with sensitivity checks. Rolling analysis uses 2,000 replicates and six-month blocks. Every portfolio receives the same sampled month indices, preserving cross-model co-movement. Model-gap intervals are the direct evidence for comparisons.

8. Findings, interpretation and reproducible use

LSTM has a higher observed full-test monthly Sharpe than Transformer, a gap of approximately 0.155. Its basic paired interval $[-0.049, 0.365]$ includes zero, so the full-test advantage remains statistically unresolved. Possible explanations for Transformer performance include limited macro history and training or tuning needs, but the completed experiment does not establish these causes.

The equal-weight blend lowers monthly volatility to about 1.01 percent because LSTM and Transformer returns correlate only about 0.29. Its Sharpe gain over LSTM is approximately 0.004, which remains unresolved. The blend comparison is test-informed and needs a fresh holdout. It averages returns, rather than retraining a joint model.

Recession performance relies on only 26 observations. Higher observed recession Sharpe does not establish a stable advantage. Rolling-window verdicts are dependent because windows overlap. Individual confidence intervals differ from paired gap intervals. Pointwise rolling bands differ from simultaneous intervals calibrated for a family of model pairs.

Reproducibility map: notebooks 01-06 build data and replicated models; Code 07 compares replication results; Code 08 implements the matched architecture experiment; Codes 09-012 evaluate the saved results. The completed matched configuration is saved in `results/paper_matched_reduced_schedule/manifest.json`. Core matched training is implemented in `src/models/transformer_gan_paper.py`. Presentation tables and figures summarize these saved outputs.

Next steps are a completed full-schedule matched run, a fresh holdout, turnover and transaction-cost measurement, and uncertainty across independent training runs. Fixed-return bootstrap intervals quantify historical sampling uncertainty, not every source of learning or selection uncertainty.

Repository:

<https://github.com/rezazamani2329/Deep-Learning-in-Asset-Pricing-Replication-and-Extensions>

This document describes the implemented project and saved configuration. It distinguishes proposed work from completed results.